

Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Сибирский государственный университет телекоммуникаций и
информатики»
(СибГУТИ)

Кафедра ТСиВС

ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ

**По дисциплине: «Интернет-технологии и сетевое
программное обеспечение»**

**Тема: «Разработка финального проекта: Система управления
счетами (invoices-s13)»**

Студент группы ИКС-433

ФИО студента Криволапов Н. А.

Проверил:

Должность ст. преподаватель

ФИО Лошкарев А. А.

Новосибирск 2026 г.

Содержание

ВВЕДЕНИЕ	2
1 ПОСТАНОВКА ЗАДАЧИ (INVOICES-S13)	3
1.1 Назначение информационной системы	3
1.2 Функциональные требования к системе	3
1.3 Нефункциональные требования	3
1.4 Спецификация варианта s13	3
2 АРХИТЕКТУРА СИСТЕМЫ УПРАВЛЕНИЯ СЧЕТАМИ	4
2.1 Компонентная структура	4
2.2 стек протоколов взаимодействия	4
2.3 Управление данными и жизненный цикл	4
2.3.1 Алгоритм запуска и обнаружения сервисов	4
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КОМПОНЕНТОВ	5
3.1 Реализация gRPC-контракта и сервера	5
3.2 Реализация REST-шлюза на FastAPI	5
3.3 Система обработки исключительных ситуаций	5
4 КОНТЕЙНЕРИЗАЦИЯ, ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ	6
4.1 Спецификация контейнеров (Docker)	6
4.2 Сценарии верификации работоспособности	6
4.3 Автоматизация сборки и тестирования	6
4.4 Ограничения и потенциал развития	6
ЗАКЛЮЧЕНИЕ	7

ВВЕДЕНИЕ

Современные распределенные системы часто строятся по принципу разделения ответственности между независимыми компонентами. Это позволяет эффективно масштабировать отдельные части системы, использовать наиболее подходящие технологии для каждой задачи и обеспечивать высокую отказоустойчивость.

В рамках данной работы разработана учебная распределенная система управления счетами `invoices-s13` на языке Python. Система позволяет регистрировать финансовые документы, получать информацию об их текущем статусе и суммах, а также управлять списком доступных документов через единый интерфейс. Разделение обязанностей между gRPC-сервисом бэкенда и REST-шлюзом обеспечивает гибкость API и высокую производительность внутреннего взаимодействия.

1 ПОСТАНОВКА ЗАДАЧИ (INVOICES-S13)

1.1 Назначение информационной системы

Информационная система `invoices-s13` разработана для автоматизации учета финансовых документов (счетов) в рамках микросервисной архитектуры. Основная цель системы --- обеспечить надежное хранение данных о счетах и предоставить гибкий интерфейс для их создания и получения внешними потребителями.

1.2 Функциональные требования к системе

Система должна реализовывать следующий набор функций:

- Создание записи о счете с обязательным указанием суммы (`amount`).
- Получение подробной информации по идентификатору счета.
- Формирование списка всех существующих счетов через потоковую передачу данных.
- Корректная трансляция данных между различными форматами представления (Protobuf и JSON).

1.3 Нефункциональные требования

- **Стек технологий:** Использование Python для всех компонентов системы.
- **Взаимодействие:** Применение gRPC для внутренних вызовов и REST для внешних.
- **Изоляция:** Каждый компонент должен функционировать в собственном Docker-контейнере.

1.4 Спецификация варианта s13

Согласно заданию, система работает с объектом `invoice`. Основной числовой атрибут --- `amount` (тип `float`). Все компоненты системы используют идентификатор `s13` в названиях и конфигурациях.

2 АРХИТЕКТУРА СИСТЕМЫ УПРАВЛЕНИЯ СЧЕТАМИ

2.1 Компонентная структура и распределение ролей

Архитектура информационной системы `invoices-s13` базируется на паттерне <<API Gateway>>, который является стандартом для современных микросервисных приложений. Система разделена на два функциональных контура:

1. **API Gateway (Внешний контур):** Выступает в роли единой точки входа (Single Point of Entry). Его основные задачи включают маршрутизацию запросов, агрегацию данных из различных сервисов (в перспективе), выполнение функций обратного прокси и трансляцию протоколов (Protocol Translation) из внешнего REST/JSON во внутренний gRPC/Protobuf. Использование шлюза позволяет скрыть детали реализации бэкенда от клиента.
2. **Invoices Service (Внутренний контур):** Является <<сервисом-источником истины>> для финансовых документов. Он инкапсулирует всю бизнес-логику: валидацию сумм (`amount`), генерацию идентификаторов и управление состоянием счетов. Сервис спроектирован так, чтобы быть максимально изолированным и независимым от способа отображения данных клиенту.

2.2 Стек протоколов и обоснование выбора

Для обеспечения баланса между доступностью и производительностью в системе реализован гибридный подход к коммуникациям:

- **REST (HTTP/1.1 + JSON):** Используется для взаимодействия <<клиент-шлюз>>. Выбор обусловлен широкой поддержкой протокола всеми современными языками программирования и браузерами. JSON обеспечивает легкость отладки и читаемость сообщений.
- **gRPC (HTTP/2 + Protobuf):** Используется для взаимодействия <<шлюз-сервис>>. В отличие от REST, gRPC использует бинарную сериализа-

цию Protobuf, что существенно (в 3-5 раз) уменьшает размер передаваемого трафика. Использование HTTP/2 позволяет мультиплексировать несколько запросов в одном TCP-соединении, что критично для высоконагруженных распределенных систем.

2.3 Модель управления данными и жизненный цикл

В текущей реализации используется оперативная память (in-memory) для хранения объектов `invoice`. Это решение позволяет минимизировать время отклика системы на этапе тестирования. Данные организованы в виде потокобезопасного словаря, где ключом выступает строковый ID.

Архитектура системы поддерживает горизонтальное масштабирование: при увеличении нагрузки можно запустить несколько экземпляров `invoices-svc-s1`. Однако, для корректной работы в таком режиме потребуется внедрение распределенной базы данных (например, PostgreSQL или Redis), что предусмотрено интерфейсом репозитория в коде.

2.4 Оркестрация и Service Discovery

Взаимодействие компонентов в динамической среде Docker обеспечивается встроенным механизмом Service Discovery. Шлюз не использует статические IP-адреса, а обращается к бэкенду по его логическому имени `invoices-svc-s13`. Внутренняя DNS-служба Docker разрешает это имя в актуальный IP-адрес контейнера, что позволяет прозрачно обновлять и перезапускать сервисы.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КОМПОНЕНТОВ

3.1 Реализация gRPC-контракта и сервера

Сердцем системы является файл `invoices.proto`, определяющий структуру сообщений и методы сервиса. Сервер реализует эти методы, обеспечивая обработку запросов.

Listing 1: Метод `CreateInvoice` в `Invoices Service`

```
1 def CreateInvoice(self, request, context):
2     new_id = str(len(self._invoices) + 1)
3     invoice = invoices_pb2.Invoice(id=new_id, name=request.name,
4                                     amount=request.amount)
5     self._invoices[new_id] = invoice
6     return invoices_pb2.CreateInvoiceResponse(invoice=invoice)
```

3.2 Реализация REST-шлюза на FastAPI

Шлюз выполняет роль адаптера. Он принимает POST и GET запросы, формирует соответствующие gRPC сообщения и возвращает результат клиенту.

Listing 2: Эндпоинт для создания счета в `Gateway`

```
1 @app.post("/api/invoices", response_model=InvoiceSchema,
2           status_code=201)
3 async def create_invoice(invoice: CreateInvoiceSchema):
4     client = get_grpc_client()
5     response = client.CreateInvoice(invoices_pb2.
6                                     CreateInvoiceRequest(
7                                         name=invoice.name, amount=invoice.amount
8                                     ))
9     return InvoiceSchema(id=response.invoice.id, name=response.
10                          invoice.name, amount=response.invoice.amount)
```

3.3 Система обработки исключительных ситуаций

В шлюзе реализован механизм перехвата ошибок gRPC (например, `StatusCode` и их преобразования в соответствующие HTTP-статусы (404), что обеспечивает корректное поведение API.

4 КОНТЕЙНЕРИЗАЦИЯ, ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ

4.1 Спецификация контейнеров (Docker)

Для каждого микросервиса создан легковесный образ на базе Python. Это гарантирует, что система будет работать одинаково на любой машине, где установлен Docker.

4.2 Сценарии верификации работоспособности

Проверка системы осуществляется через последовательность действий:

1. Сборка и запуск: `docker-compose up --build`.
2. Создание объекта: запрос `POST /api/invoices`.
3. Чтение объекта: запрос `GET /api/invoices/1`.
4. Проверка списка: запрос `GET /api/invoices`.

4.3 Автоматизация сборки и тестирования

Сборка проекта и запуск тестов автоматизированы с помощью `Makefile`. Тесты проверяют наличие всех необходимых файлов и корректность указания `project_code` (`invoices-s13`) в документации.

4.4 Ограничения и потенциал развития

На текущем этапе система является функциональным прототипом. В качестве дальнейшего развития планируется внедрение персистентной БД (PostgreSQL) и системы аутентификации на базе JWT-токенов.

ЗАКЛЮЧЕНИЕ

В результате выполнения работы была спроектирована и реализована распределенная система управления счетами `invoices-s13`. Основной акцент был сделан на использовании современных технологий межсервисного взаимодействия и обеспечении прозрачности API для внешних потребителей.

Были успешно решены задачи по проектированию gRPC-контракта, реализации шлюза на FastAPI и контейнеризации приложения. Полученные в ходе работы навыки проектирования микросервисной архитектуры являются базой для разработки масштабируемых промышленных приложений.